

QCM Dates et fuseaux

20 questions /20



Avant de raisonner en fuseaux et en périodes glissantes. Durée conseillée : 20 min
· 3 paliers · corrigé détaillé.

Objectif : mesurer tes réflexes réels face aux dates, pas ta connaissance des noms de fonctions. Ce test vérifie si tu sais quand un fuseau décale un jour calendaire, quelle borne un `DATEDIFF` ou un `BETWEEN` inclut réellement, et pourquoi un total « à date » peut mentir sans qu'aucune requête ne plante.

J'ai déjà vu un dashboard afficher un lundi matin à zéro euro de CA, simplement parce que le job de calcul tournait sur un serveur en UTC pendant que toute l'équipe raisonnait en heure de Paris.

Repères : 12/20 = les bases sont posées, 17/20 = profil qu'on laisse toucher un pipeline multi-fuseaux sans supervision.

Candidat _____ Date _____ Note _____ /20

Palier 1 : les bases qui évitent la honte en debug.

Six questions sur le socle : UTC contre heure locale, stockage, semaines et fuseau du serveur.

Question 1. Une commande est enregistrée à 23h50 heure de Los Angeles (UTC-8), puis stockée en UTC. Sur un dashboard configuré en UTC, cette commande apparaît datée :

- a) Le lendemain (jour suivant), car UTC est en avance sur Los Angeles.
- b) La veille (jour précédent).
- c) Le même jour, l'heure ne change pas le jour calendaire.
- d) Cela dépend uniquement du navigateur du client.

Question 2. Quelle est la différence essentielle entre les types `timestamp` et `timestampz` en PostgreSQL ?

- a) `timestampz` stocke un fuseau horaire nommé (ex. Europe/Paris) avec chaque valeur.
- b) `timestampz` convertit et stocke en UTC en interne, puis affiche selon le fuseau de la session; `timestamp` stocke la valeur telle quelle, sans conversion.
- c) `timestamp` ignore complètement les secondes.
- d) Les deux types sont strictement identiques en PostgreSQL.

Question 3. La norme ISO 8601 définit la semaine 1 de l'année comme :

- a) La semaine contenant le 1^{er} janvier, quel que soit le jour de la semaine.
- b) La première semaine complète de sept jours de l'année.
- c) La semaine contenant le premier jeudi de l'année (donc au moins 4 jours en janvier).
- d) La semaine qui commence un dimanche et contient le 1^{er} janvier.

Question 4. Une entreprise a une année fiscale qui débute en juillet. Un rapport « YTD » calculé avec `YEAR(date) = YEAR(TODAY())` va :

- a) Donner le bon cumul fiscal, `YEAR()` s'adapte automatiquement.
- b) Renvoyer une erreur car `YEAR()` ne supporte pas les années décalées.
- c) Fonctionner seulement si le mois de juillet est explicitement filtré ailleurs.
- d) Cumuler depuis janvier au lieu de juillet, faussant le total fiscal réel.

Question 5. Que renvoie la fonction `DAX EOMONTH(DATE(2026,2,15), 0)` ?

- a) 2026-02-28.
- b) 2026-02-01.
- c) 2026-03-01.
- d) 2026-02-15.

Question 6. Un job planifié pour s'exécuter tous les jours à minuit s'appuie sur l'horloge du serveur (UTC). Si l'équipe pense que le job tourne à minuit heure de Paris, il va en réalité s'exécuter :

- a) À minuit heure de Paris, UTC et heure locale sont toujours synchronisées.
- b) 1h ou 2h plus tard que minuit Paris, selon la saison (heure d'été ou d'hiver).
- c) Toujours 12h avant minuit Paris.
- d) De façon aléatoire, selon la charge du serveur.

Palier 2 : les pièges qui passent les tests mais pètent en prod.

Sept questions où la requête tourne sans erreur, mais le chiffre affiché est faux.

Question 7. En France, le passage à l'heure d'été fait sauter l'horloge de 2h00 à 3h00 le dernier dimanche de mars. Une valeur « 2026-03-29 02h30 » heure locale est :

- a) Une heure valide, simplement rare.
- b) Toujours interprétée comme 03h30.
- c) Une heure qui n'existe pas ce jour-là : elle a été sautée par le changement d'heure.
- d) Automatiquement convertie en UTC sans ambiguïté.

Question 8. Le dernier dimanche d'octobre, l'horloge française repasse de 3h00 à 2h00 (heure d'hiver). La valeur locale « 2026-10-25 02h30 » correspond à :

- a) Un seul instant précis, sans ambiguïté.
- b) Une erreur de saisie systématique.
- c) Toujours le second passage (heure d'hiver).
- d) Deux instants distincts possibles à une heure d'écart, car cette heure locale est vécue deux fois ce jour-là.

Question 9. `WHERE date_creation BETWEEN '2026-01-01' AND '2026-01-31'` sur une colonne `datetime`. Que rate cette requête ?

- a) Toutes les lignes du 31 janvier après 00h00, car '2026-01-31' est interprété comme minuit pile.
- b) Rien, elle couvre tout janvier.
- c) Les lignes du 1^{er} janvier avant midi.
- d) Elle provoque une erreur de syntaxe.

Question 10. `DATEDIFF(jour, '2026-01-01', '2026-01-10')` renvoie 9 jours d'écart. Un collègue calcule « 10 jours de campagne » en ajoutant 1 au résultat. Ce raisonnement est :

- a) Faux dans tous les cas, `DATEDIFF` compte toujours les deux bornes.
- b) Correct seulement si la campagne inclut ses deux dates de bornes (bornes inclusives), sinon 9 jours est la bonne mesure.
- c) Sans importance, plus ou moins un jour ne change jamais un total.
- d) Correct uniquement en année bissextile.

Question 11. Le 5 du mois, un dashboard affiche un CA « du mois en cours » calculé comme les 30 derniers jours glissants. Le problème :

- a) Aucun, rolling 30 jours et MTD donnent toujours le même chiffre.
- b) MTD est toujours plus grand que le rolling 30 jours.
- c) Le rolling 30 jours inclut une bonne partie du mois précédent, alors que MTD ne devrait cumuler que depuis le 1^{er} du mois : les deux mesures ne racontent pas la même histoire.
- d) Le rolling 30 jours ne fonctionne qu'en fin de mois.

Question 12. Une table `commandes` a une colonne `date_commande` de type `date`, une table `connexions` a un horodatage de type `datetime`. Un JOIN direct sur ces deux colonnes va :

- a) Fonctionner normalement, SQL convertit toujours implicitement `date` et `datetime` sans perte.
- b) Provoquer systématiquement une erreur de type.
- c) Ne matcher que les lignes du 1^{er} du mois.
- d) Rater les correspondances dès que l'horodatage n'est pas exactement à minuit, car `date` compare à une heure implicite 00:00:00.

Question 13. Un fichier CSV contient la date « 03/04/2026 ». Importé avec un parseur configuré en locale anglaise (MM/JJ/AAAA), cette date est lue comme :

- a) Le 4 mars 2026, alors que l'émetteur du fichier voulait dire le 3 avril.
- b) Le 3 avril 2026, la locale n'affecte pas les dates à deux chiffres.
- c) Une erreur de parsing systématique.
- d) Le 3 avril 2026 uniquement si l'année est sur 4 chiffres.

Palier 3 : le raisonnement qui distingue un stagiaire d'un analyste qu'on titularise.

Sept questions de comparaison de périodes, de modèle Power BI et de fuseaux multi-régions : là où se joue le 17/20.

Question 14. Pour comparer le CA de février à celui de janvier, le piège des mois de longueurs différentes (28 contre 31 jours) est que :

- a) Aucun piège, DAX ajuste automatiquement le nombre de jours.
- b) Comparer un total brut CA février contre CA janvier sans ramener au CA moyen par jour peut faire croire à une baisse d'activité alors que février a simplement moins de jours.
- c) SAMEPERIODYLASTYEAR ne fonctionne jamais sur des mois de longueur différente.
- d) Il faut toujours comparer semaine par semaine pour éviter le problème.

Question 15. Dans un modèle Power BI sans table de dates dédiée marquée comme telle, les fonctions de time intelligence comme `TOTALYTD` ou `DATEADD` :

- a) Fonctionnent normalement tant qu'il existe une colonne `date` quelque part dans le modèle.
- b) Sont interdites par Power BI sans licence Premium.
- c) Risquent de produire des résultats faux ou des erreurs, car elles ont besoin d'un calendrier continu, sans trou, marqué « table de dates ».
- d) Ne posent problème que si le modèle contient plus de deux tables de faits.

Question 16. Un pipeline tourne sur un serveur cloud en UTC. Un analyste filtre `WHERE date_evenement = CURRENT_DATE` en pensant obtenir « aujourd'hui » heure de Paris. Entre minuit et 2h du matin heure de Paris (été, UTC+2), ce filtre :

- a) Retourne toujours le bon jour local, `CURRENT_DATE` s'ajuste au fuseau de l'utilisateur.
- b) Provoque une erreur car `CURRENT_DATE` ne peut pas être comparé à une colonne `timestamp`.
- c) Ne pose problème qu'en hiver, jamais en été.
- d) Utilise encore la date de la veille côté serveur (UTC), alors qu'à Paris le nouveau jour a déjà commencé : les événements du tout début de journée locale sont mal classés.

Question 17. Une table de logs utilise `timestampz`, alimentée par des serveurs situés à New York, Paris et Singapour. Lors du changement d'heure (DST) à New York, la colonne `timestampz` garantit que :

- a) L'instant réel (en UTC) reste correct et comparable entre les trois régions, même si les horaires locaux affichés changent selon le fuseau de la session de lecture.
- b) Chaque ligne garde son fuseau d'origine affiché tel quel, peu importe la session.
- c) Toutes les lignes doivent être recalculées manuellement au moment du changement d'heure.
- d) Les événements de New York deviennent incomparables à ceux de Paris pendant la transition.

Question 18. Certaines années, le 31 décembre appartient déjà à la semaine 1 ISO de l'année suivante, alors qu'une semaine calendaire simple (dimanche-samedi) le classe encore dans la dernière semaine de l'année en cours. Le risque pour un reporting hebdomadaire annuel :

- a) Aucun, les deux découpages donnent toujours le même résultat.
- b) Un CA de fin décembre peut basculer dans le mauvais exercice hebdomadaire selon la convention choisie, faussant les comparaisons semaine par semaine entre systèmes.
- c) Le problème ne concerne que les années bissextiles.
- d) La semaine ISO ne dépasse jamais 52.

Question 19. Pour un CA « à date » comparable jour par jour avec l'année précédente (même nombre de jours écoulés), la mesure la plus fiable est :

- a) TOTALYTD, qui cumule automatiquement du 1^{er} janvier jusqu'à la date du jour, à comparer telle quelle avec l'année précédente même si le nombre de jours ouvrés diffère.
- b) Un rolling 30 jours glissant, plus simple à lire.
- c) TOTALYTD borné au même jour calendaire l'an dernier, en vérifiant que le nombre de jours ouvrés cumulés est comparable, pas seulement la date.
- d) Le CA du dernier jour seul, multiplié par le nombre de jours écoulés.

Question 20. Un contrôle KPI compare le CA du mois en cours à celui du mois précédent, calculé avec `CALCULATE(SUM(CA), DATEADD('Dates'[Date], -1, MONTH))` sur une table de dates dédiée. Le 30 avril, ce calcul rapporte le CA :

- a) Du 30 mars uniquement.
- b) Du 1^{er} avril au 30 avril de l'année précédente.
- c) Toujours 0, DATEADD ne fonctionne pas avec MONTH.
- d) De tout le mois de mars (DATEADD décale l'ensemble du filtre de dates d'un mois, pas seulement le dernier jour).

Solutions et explications.

Un point par bonne réponse. Compte ton score, puis relis surtout les questions que tu ne peux pas expliquer à voix haute.

1. **a)** Los Angeles est en UTC-8 : 23h50 local devient 07h50 UTC le lendemain. Le piège classique est de croire que seule l'heure change, jamais le jour calendaire.
2. **b)** `timestampz` normalise en UTC à l'écriture et reconvertit à la lecture selon le fuseau de session ; `timestamp` n'effectue aucune conversion, il stocke l'heure du wall clock telle quelle.
3. **c)** La règle ISO 8601 se calque sur le premier jeudi de l'année, pas sur le 1^{er} janvier : c'est pourquoi le 1^{er} janvier peut appartenir à la semaine 52 ou 53 de l'année précédente.
4. **d)** `YEAR()` découpe toujours par année civile ; sur une année fiscale décalée, il faut construire une colonne « exercice fiscal » dédiée dans la table de dates, sinon le cumul démarre au mauvais mois.
5. **a)** 2026 n'est pas bissextile (non divisible par 4), donc février compte 28 jours : `EOMONTH(., 0)` renvoie le dernier jour du mois courant, ici le 28.
6. **b)** Le serveur exécute à minuit UTC. Selon la saison, minuit UTC vaut 1h ou 2h du matin à Paris : le job ne tourne jamais à minuit local sans conversion explicite du planificateur.
7. **c)** Le passage à l'heure d'été supprime l'intervalle 02h00-03h00 : cette plage horaire locale n'a physiquement jamais existé ce jour-là.
8. **d)** Le passage à l'heure d'hiver répète l'intervalle 02h00-03h00 : sans indicateur de décalage explicite, l'heure locale seule ne suffit pas à identifier l'instant réel.
9. **a)** '2026-01-31' sans heure vaut minuit pile en datetime : toute ligne après 00h00 ce jour-là tombe hors de la borne haute. Il faut une borne exclusive au 1^{er} février ou tronquer les deux côtés en date.
10. **b)** `DATEDIFF` compte des intervalles, pas des jours calendaires inclus : 9 jours d'écart deviennent 10 jours de campagne seulement si les deux dates de bornes sont comptées comme journées pleines.
11. **c)** Un rolling 30 jours capte une partie du mois précédent dès le début du mois ; un vrai MTD ne cumule que depuis le 1^{er}. Les confondre fausse toute lecture de tendance en début de mois.
12. **d)** Une colonne date se comporte comme un datetime à 00:00:00 : la jointure ne matche que les horodatages tombant pile à minuit, ratant la quasi-totalité des lignes réelles.
13. **a)** En locale anglaise, le premier groupe de chiffres est le mois : 03/04 devient le 4 mars, pas le 3 avril. Sans normalisation ISO (AAAA-MM-JJ) à l'import, l'ambiguïté est silencieuse.
14. **b)** Comparer des totaux bruts entre mois de longueurs différentes confond volume et rythme d'activité ; un CA moyen par jour ouvré neutralise le nombre de jours et révèle la vraie tendance.
15. **c)** Les fonctions de time intelligence s'appuient sur une table calendrier continue marquée « table de dates » : sans elle, les agrégations glissantes peuvent sauter des jours ou renvoyer des erreurs.
16. **d)** Entre minuit et 2h heure de Paris en été, l'horloge UTC du serveur affiche encore la veille : `CURRENT_DATE` calculé côté serveur classe alors les tout premiers événements du jour local dans le mauvais jour.
17. **a)** `timestampz` ancre chaque valeur sur un instant UTC unique : les trois régions restent comparables entre elles pendant une transition DST, seul l'affichage local change selon la session.
18. **b)** La semaine ISO peut placer le 31 décembre dans la semaine 1 de l'année suivante alors qu'une semaine calendaire dimanche-samedi le garde dans l'année en cours : un même chiffre de CA change d'exercice hebdomadaire selon la convention.
19. **c)** Comparer deux `TOTALYTD` au même jour calendaire ignore que le nombre de jours ouvrés écoulés peut différer d'une année sur l'autre (jours fériés mobiles, week-ends décalés) ; il faut vérifier ce nombre avant de conclure à une hausse ou une baisse.
20. **d)** `DATEADD` décale l'intégralité de la plage de filtre d'un mois, pas une seule date : le 30 avril, la mesure porte sur tout le mois de mars, ce qui est le comportement voulu pour comparer mois contre mois.

Fait main, en solo.

Si ce test t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

